
SOLO DEEP-LEARNING PORTFOLIO PROJECT

ChronoPDE

Boundary-Aware Continuous-Time Neural Operator
for Irregular Reaction-Diffusion Dynamics

Core signal PDEs + calculus + spectral operators	Training From scratch in PyTorch
Evaluation OOD + long horizon + sparse time	Delivery Repository + report + live simulator

Prepared for: Madhav Kapoor

Purpose: Replace StellarNet with one technically serious deep-learning project for highly selective UK MSc AI/ML applications

Target completion: 10-12 weeks

00. Executive Decision and Document Map

Decision
Build ChronoPDE as a complete scientific-ML system. The headline contribution is a continuous-time neural operator with a discrete cosine spectral backbone matched to Neumann boundary conditions. The project is evaluated against controlled autoregressive and Fourier baselines under sparse time sampling, long rollouts, and held-out physical coefficients.

Section	Focus	Output
01	Project charter	Scope, admissions value, success criteria
02	Mathematical problem	Reaction-diffusion dynamics and boundary conditions
03	System architecture	Splines, flow matching, DCT operator, RK4
04	Data protocol	Generation, parameter ranges, splits, leakage controls
05	Baselines and experiments	Controlled comparisons and ablations
06	Metrics and analysis	Field, spectral, physical, stability, efficiency
07	Implementation blueprint	Repository, modules, tests, configurations
08	Twelve-week execution plan	Milestones, gates, weekly outputs
09	Compute and reproducibility	Budgets, seeds, logging, artifacts
10	Risk and failure pivots	Fallbacks that preserve scientific value
11	Portfolio packaging	Demo, README, report, resume and SOP language
12	Reference specification	Hyperparameters, experiment IDs, sources

Non-negotiable principle

This is a portfolio engineering and experimentation project, not an attempt to claim a new state-of-the-art algorithm. Its strength comes from correct mathematical choices, a from-scratch implementation, clean controlled experiments, and unusually strong documentation for an undergraduate project.

01. Project Charter

What will be built, why it matters, and where the scope stops.

One-sentence specification

Build target
Train a parameter-conditioned continuous-time operator to predict the velocity field of a coupled 2D reaction-diffusion system from irregular observations, integrate the learned field at arbitrary query times, and benchmark it against autoregressive simulators on in-distribution and out-of-distribution regimes.

Admissions value for this specific profile

Existing strength	What ChronoPDE adds
LLM applications, orchestration and RAG	A fully trained numerical deep-learning model with no external model API
Backend and product engineering	Scientific computation, differential equations and operator learning
Workshop/shared-task research exposure	Independent ownership of a complete experimental system
Conventional 1D CNN project	Continuous dynamics, spectral mathematics, OOD testing and long-horizon analysis

Definition of done

- Model: DCT-based continuous-time neural operator trains successfully from scratch and generates stable rollouts.
- Evidence: At least three baselines, three important random seeds, and predetermined ID/OOD splits are evaluated.
- Artifact: Public repository, reproducible Colab notebook, model checkpoint, benchmark tables, technical report and live demo.
- Honesty: Claims are limited to measured outcomes; negative or mixed findings are documented.

Scope boundaries

In scope	Out of scope
One 2D coupled PDE system	Multi-physics foundation model
64 x 64 headline resolution	Full-scale turbulent 3D CFD
Parameter, initial-condition and temporal OOD	Test-time adaptation or mixture-of-experts
DCT/FFT spectral comparison	Novel numerical-solver theory
Small interactive simulator	Production cloud platform

02. Mathematical Problem

The physical system, continuous-time learning target, and spectral rationale.

Governing equations

$$\begin{aligned} du/dt &= D_u * \text{Laplacian}(u) + u - u^3 - k - v \\ dv/dt &= D_v * \text{Laplacian}(v) + u - v \\ \text{State: } x(t) &= [u(t, x, y), v(t, x, y)] \\ \text{Boundary: } \text{normal_derivative}(u) &= \text{normal_derivative}(v) = 0 \end{aligned}$$

Here u is the activator field, v is the inhibitor field, D_u and D_v control diffusion, and k shifts the nonlinear reaction. Random initial conditions evolve into structured spatial patterns. The nonlinear coupling makes this considerably richer than one-step image prediction.

Why use the discrete cosine transform?

- The Laplacian with homogeneous Neumann boundaries has cosine eigenfunctions.
- A DCT spectral layer therefore represents spatial dynamics in a basis aligned with zero normal flux.
- An FFT-based FNO is a strong baseline, but its periodic basis is not naturally matched to this boundary condition.
- The DCT-versus-FFT comparison turns boundary treatment into a controlled mathematical experiment rather than decorative "physics awareness."

Continuous-time learning target

Given irregular observations $x(t_i)$, fit a local spline path $\mu(t)$. Sample a continuous time τ inside an observed interval, evaluate the interpolated state $\mu(\tau)$, and compute its analytic derivative $d\mu/dt$. Train a neural operator v_θ to regress that derivative:

Core objective

$L_{\text{velocity}} = \text{mean} || v_\theta(\mu(\tau), \tau, p) - d\mu(\tau)/dt ||^2$. A small smooth perturbation may be added to the spline path as a regularizer. No gradients are propagated through the numerical integrator during this stage.

Falsifiable hypothesis

Under matched parameter count and trajectory budget, continuous-time training will outperform autoregressive training as observations become temporally sparse or irregular, while the DCT backbone will reduce boundary-flux and spectral errors relative to a continuous-time FFT backbone.

03. System Architecture

A compact model with enough depth to demonstrate real architecture work.

TRAINING: learn a continuous vector field without differentiating through the solver



Conditioning $p = [D_u, D_v, k]$ enters each spectral block through FiLM.
The DCT basis is aligned with zero-normal-flux Neumann boundaries; FFT-FNO remains a controlled ablation.
INFERENCE: choose Euler, Heun, or RK4 and sweep function evaluations to obtain an accuracy-cost curve.

Component	Specification	Reason
Input	Two fields u,v ; normalized time t ; parameters D_u,D_v,k	Represents state, time and dynamics regime
Lift	1×1 convolution, $2 \rightarrow$ width 32 or 48	Moves physical fields to latent channels
Conditioner	MLP over $[t,D_u,D_v,k]$ producing FiLM scales and biases	Allows one model to represent a family of PDEs
Spectral block	2D DCT, learned low-mode channel mixing, inverse DCT, residual pointwise path, GELU	Global interactions with boundary-aligned basis
Depth	Four spectral blocks	Enough capacity without excessive Colab cost
Projection	1×1 convolution, width $\rightarrow 2$	Predicts du/dt and dv/dt
Integrator	Euler, Heun and RK4 implementations	Arbitrary-time inference and accuracy-cost analysis

Recommended training objective

Term	Definition	Default
Velocity loss	Physical-space MSE between predicted and spline-derived velocity	1.0
Spectral loss	Relative error over retained DCT coefficients; report low/high bands separately	0.05
Optional short-rollout loss	One or two RK4 segments, enabled only after core training is stable	0.0 initially

Implementation constraint
Write the DCT layer, FiLM conditioning, splines and fixed-step integrators yourself in PyTorch. External libraries may be used for data loading, logging and numerical-reference tests. The repository must clearly identify inspiration from CFO and distinguish reproduction from your boundary-aware extension.

03A. Training and Inference Algorithms

Flow-matched velocity training

```
TRAINING STEP
1 trajectory, parameters, observed_times <- batch
2 interval <- sample adjacent observed times
3 tau <- Uniform(interval.start, interval.end)
4 x_tau, dx_tau <- evaluate_spline_and_derivative(tau)
5 dx_hat <- dct_operator(x_tau, tau, parameters)
6 loss <- velocity_mse(dx_hat, dx_tau) + 0.05 * spectral_loss
7 update model parameters with AdamW
```

Fixed-step RK4 rollout

```
INFERENCE
1 x <- initial_state; t <- start_time
2 while t < query_time:
3   k1 <- v_theta(x, t, p)
4   k2 <- v_theta(x + dt*k1/2, t + dt/2, p)
5   k3 <- v_theta(x + dt*k2/2, t + dt/2, p)
6   k4 <- v_theta(x + dt*k3, t + dt, p)
7   x <- x + dt*(k1 + 2*k2 + 2*k3 + k4)/6
8   t <- t + dt
9 return x
```

Validation tests before any large run

Test	Expected condition
DCT round-trip	$\max_{\text{abs}}(x - \text{IDCT}(\text{DCT}(x))) < 1\text{e-}5$ in float32
DCT reference	coefficients agree with SciPy on small random tensors
RK4 convergence	error decreases at approximately fourth order on a known ODE
Spline derivative	analytic derivative agrees with finite difference on smooth test functions
Boundary derivative	cosine-reconstructed fields have near-zero normal derivative
Overfit smoke test	model overfits 2-4 trajectories before full training

04. Data Generation and Split Protocol

A compact parametric dataset designed for clean OOD claims.

Dataset choice

Use the PDEBench 2D FitzHugh-Nagumo reaction-diffusion formulation and generation code. Generate a custom 64 x 64 dataset so coefficient ranges, seeds and OOD boundaries are known exactly. The published 128 x 128 data may be used for validation, but should not define the only experiment.

Split	Trajectories	Coefficient regime	Initial conditions	Purpose
Train	320	Central ranges	Smooth random fields; train frequency band	Optimization
Validation	60	Central ranges, unseen values	Unseen seeds	Early stopping/tuning
ID test	100	Same ranges, unseen values	Unseen seeds	Standard generalization
OOD parameter	120	Outside train ranges	Train-like spectra	Coefficient extrapolation
OOD initial condition	120	Central ranges	Higher spatial frequencies/amplitudes	State-distribution shift

Starting coefficient ranges

Parameter	Training range	Validation/ID	OOD range
D_u	[0.8, 1.2] x 10 ⁻³	interior unseen values	[0.6, 0.75] or [1.25, 1.4] x 10 ⁻³
D_v	[4.0, 6.0] x 10 ⁻³	interior unseen values	[3.0, 3.8] or [6.2, 7.0] x 10 ⁻³
k	[0.004, 0.006]	interior unseen values	[0.0025, 0.0038] or [0.0062, 0.0075]

These are initial design ranges, not promises. Run a small numerical sweep first and narrow any range that causes qualitatively incompatible or unstable solver behavior. Save exact values per trajectory.

Temporal observation protocol

- Full: retain every stored time point.
- 50% irregular: retain the initial/final states and randomly retain half of interior points per trajectory.
- 25% irregular: retain the initial/final states and randomly retain one quarter of interior points.
- Unseen-time evaluation: evaluate at hidden midpoints and on a denser query grid using complete numerical ground truth.

Leakage prevention
Split at the trajectory level before creating time intervals. No states from the same simulated trajectory may appear in both training and evaluation. Normalize with training-set statistics only. Freeze OOD ranges and test seeds before final tuning.

05. Baselines, Ablations and Experiment Matrix

Every comparison isolates a specific claim.

ID	Model	Time formulation	Spatial backbone	What it tests
B1	AR U-Net	Fixed one-step	Convolutions	Strong local inductive-bias baseline
B2	AR FNO	Fixed one-step	FFT spectral	Standard neural-operator baseline
B3	CT-FFT	Spline velocity + RK4	FFT spectral	Isolates benefit of continuous time
M	ChronoPDE	Spline velocity + RK4	DCT spectral	Full boundary-aware model

Fairness controls

- Match model parameters within approximately 10%; report exact counts.
- Use the same training trajectories, normalization, optimizer family and early-stopping budget.
- Tune only on validation data. Do not select methods based on OOD test performance.
- Report three seeds for the headline comparison and confidence intervals or standard deviations.
- Compare both equal wall-clock budget and equal number of training examples where practical.

Minimum experiment matrix

Experiment	Models	Shift	Primary readout
E01 ID rollout	B1, B2, B3, M	None	nRMSE vs time
E02 sparse-50	B1, B2, B3, M	50% irregular time	trajectory nRMSE
E03 sparse-25	B1, B2, B3, M	25% irregular time	trajectory nRMSE
E04 arbitrary time	B3, M	hidden query times	interpolation error
E05 parameter OOD	B1, B2, B3, M	D_u, D_v, k	OOD/ID error ratio
E06 IC OOD	B1, B2, B3, M	frequency/amplitude	spectral error
E07 basis ablation	B3 vs M	Neumann boundary	boundary flux
E08 integrator sweep	M	Euler/Heun/RK4; NFE	accuracy-cost curve
E09 mode sweep	M	8/12/16/20 modes	error vs parameters/runtime

Claim discipline
ChronoPDE need not win every cell. The expected headline is narrower: better behavior under irregular timing, arbitrary queries and long rollouts, plus lower boundary error from the DCT basis. Do not rewrite the project around whichever metric happens to win.

06. Metrics and Analysis Protocol

Measure accuracy, physics, stability and efficiency - not one aggregate score.

Category	Metric	Interpretation
Field accuracy	Relative L2 and normalized RMSE	Overall state reconstruction quality
Temporal stability	Error-vs-time curve; final-time error	How rollout error accumulates
Correlation horizon	First time correlation drops below a fixed threshold	Useful when precise trajectories diverge
Spectral fidelity	Relative error in low and high spatial-frequency bands	Whether patterns and sharp structures are preserved
Boundary physics	Mean squared normal derivative on all four boundaries	Violation of no-flow Neumann condition
OOD robustness	OOD/ID error ratio and worst-group error	Sensitivity to physical distribution shift
Efficiency	Latency, peak memory, parameter count and NFE	Cost of continuous-time inference

Statistical reporting

- Use per-trajectory metrics; report mean, median and bootstrap 95% confidence intervals.
- For three-seed headline runs, show mean plus standard deviation across seeds.
- Use paired comparisons because every model is evaluated on the same test trajectories.
- Include at least one failure-case panel, not only the best qualitative sample.
- Archive raw per-trajectory scores as CSV so every figure can be regenerated.

Required figures

Figure	Content
F1	Ground truth, ChronoPDE, strongest baseline and absolute error at four rollout times
F2	Relative L2 error versus time for ID and parameter-OOD tests
F3	Performance versus retained temporal observations: 100%, 50%, 25%
F4	DCT-versus-FFT boundary flux and spectral-band error
F5	Accuracy versus NFE/runtime for Euler, Heun and RK4
F6	OOD regime heatmap over D_u and D_v

Best single demo view
A synchronized four-panel animation: ground truth, ChronoPDE, strongest baseline and absolute error. Controls select D_u, D_v, k, initial-condition seed, query time and solver step budget.

07. Implementation Blueprint

Repository layout, interfaces and engineering standards.

```
chronopde/  
configs/ # model, data and experiment YAMLS  
chronopde/  
data/ # simulation, masks, splits, normalization  
numerics/ # DCT, splines, finite differences, integrators  
models/ # DCT operator, FNO, U-Net, conditioning  
training/ # losses, trainer, checkpoints, logging  
evaluation/ # metrics, rollout, OOD and statistical analysis  
visualization/ # field plots, animations and dashboards  
scripts/ # generate_data, train, evaluate, export_demo  
tests/ # transforms, solvers, splits and smoke tests  
demo/ # Streamlit or Gradio application  
notebooks/ # one clean reproducible Colab notebook  
reports/ # tables, figures and technical report source  
README.md # claims, quickstart, results and limitations
```

Command-line contract

Command	Expected output
python scripts/generate_data.py --config configs/data.yaml	Versioned dataset plus metadata manifest
python scripts/train.py model=chronopde seed=0	Checkpoint, resolved config and training curves
python scripts/evaluate.py experiment=ood_params	Raw scores, aggregate JSON, tables and figures
pytest -q	Transform, solver, split and smoke-test validation
streamlit run demo/app.py	Interactive local simulator

Engineering quality gates

- Typed configuration objects or validated YAML schema.
- Deterministic trajectory split manifests committed to the repository.
- Automatic checkpoint resume for interrupted Colab sessions.
- Mixed precision only after numerical correctness is established.
- Every result table generated from saved metrics, never typed manually.
- README includes tested environment, hardware, runtime and exact reproduction commands.

08. Twelve-Week Execution Plan

Each week ends with a concrete artifact and a go/no-go gate.

Week	Focus	Required output	Gate
1	PDE and numerical setup	Equation note; solver runs; dataset samples	Patterns are stable and interpretable
2	Data pipeline	64 x 64 dataset; manifests; temporal masks	No trajectory leakage
3	Numerical primitives	DCT, spline and RK4 tests	All numerical tests pass
4	Autoregressive baselines	U-Net and FFT-FNO ID curves	Both overfit smoke set and train normally
5	Continuous-time FFT	Spline velocity training; RK4 rollout	Beats trivial persistence baseline
6	ChronoPDE model	DCT operator and FiLM conditioning	Stable ID rollout
7	Temporal experiments	100/50/25% and arbitrary-time results	Fixed protocol executes end to end
8	OOD experiments	Parameter and initial-condition shifts	Frozen OOD test suite
9	Ablations and efficiency	Basis, modes, integrator and NFE sweeps	Claims have controls
10	Replication	Three-seed headline results; confidence intervals	Numbers are stable enough to report
11	Portfolio layer	Demo, animations, README and model card	Fresh-machine quickstart works
12	Final communication	6-8 page report; resume bullets; demo video	Release candidate tagged

Weekly operating rhythm

Cadence	Action
Monday	Set one measurable target and freeze the experiment configuration.
Tuesday-Thursday	Implement or run experiments; save raw outputs continuously.
Friday	Analyze failures and update the lab notebook with evidence.
Weekend	Clean code, regenerate figures and record a two-minute progress demo.

Stop rule
If the full ChronoPDE model cannot stably overfit four trajectories by the end of Week 6, pause all OOD work. Debug normalization, spline derivatives, time scaling and integrator consistency before adding new ideas.

09. Compute Budget and Reproducibility

Designed for a MacBook plus multiple Colab sessions.

Workload	T4 estimate	A100 estimate	Notes
Data generation	5-15 CPU hours	Same	Can run locally or on Colab CPU
Smoke/overfit tests	< 1 GPU hour total	< 0.5 hour	Use tiny subsets
One full 64 x 64 run	3-7 GPU hours	1.5-3 GPU hours	Width 32-48; early stopping
Baseline development	8-15 GPU hours	4-8 GPU hours	Reuse loaders and evaluation
Final three-seed suite	25-45 GPU hours	10-22 GPU hours	Only selected configurations
Total expected	35-65 GPU hours	15-35 GPU hours	Spread across 10-12 weeks

Default model configuration

Hyperparameter	Value
Item	Default
Resolution	64 x 64
State channels	2
Latent width	32
Spectral modes	12 x 12
Operator blocks	4
Batch size	8-16
Optimizer	AdamW
Initial learning rate	3e-4
Scheduler	Cosine decay with short warmup
Precision	float32 initially; AMP after validation
Gradient clipping	1.0
Early stopping	Validation rollout nRMSE

Reproducibility checklist

- Record Python, PyTorch, CUDA and GPU versions.
- Store data-generation parameters, simulation code commit and trajectory seeds.
- Log resolved configuration, git commit, parameter count and hardware for every run.
- Use seed 0 for development; run seeds 0, 1 and 2 only after configurations are frozen.
- Upload final checkpoints and a lightweight sample dataset; provide a full-data generation command.

10. Risk Register and Failure Pivots

A failed hypothesis must still produce a strong, honest technical artifact.

Risk	Early warning	Mitigation / pivot
Spline targets are noisy	Velocity loss falls but rollout diverges	Rescale time; use quintic Hermite splines; smooth derivative estimates; compare linear path
DCT gives no global nRMSE gain	FFT and DCT overlap on ID curves	Focus on boundary flux and high-frequency fidelity; report when basis alignment matters
Continuous time loses on short horizons	AR baselines lead at first steps	Analyze crossover horizon and sparse-time regime; do not claim universal superiority
Coefficient OOD is too severe	All methods fail catastrophically	Introduce mild/medium/severe tiers and report degradation curves
RK4 inference is slow	Good error but poor latency	Sweep Euler/Heun/RK4 and step counts; present Pareto frontier
Colab sessions interrupt	Lost runs or incomplete logs	Checkpoint each epoch; store metrics incrementally; resume deterministically
Model instability	NaNs or exploding fields	Clip gradients; normalize channels; lower dt; validate vector-field magnitude
Schedule slips	Core model incomplete after Week 6	Drop optional rollout loss and 128 x 128 transfer; preserve main experiment

Scientifically meaningful negative-result report

If ChronoPDE does not dominate the baselines, frame the final report around a regime map. Show how performance changes with temporal sparsity, coefficient distance, initial-condition frequency and solver budget. Analyze whether error originates in spline derivative estimation, vector-field approximation, integration, spectral truncation or boundary mismatch. This demonstrates mature experimental reasoning even without a headline win.

Minimum successful result
A reproducible system plus one well-supported conditional conclusion, such as: "continuous-time training improves forecasts under 25% irregular observations," or "the cosine basis reduces boundary violations without improving global nRMSE." Conditional findings are legitimate; unsupported universal claims are not.

Features to cut first if time is limited

- 128 x 128 resolution-transfer experiment.
- Optional short-rollout training loss.
- Large hyperparameter search.
- Multiple demo frameworks or cloud deployment.
- Any test-time adaptation, uncertainty routing or mixture-of-experts addition.

11. Portfolio Packaging and Admissions Positioning

The finished artifact must be understandable in ninety seconds and defensible in a technical interview.

Artifact	Minimum standard
GitHub repository	Clean architecture, tests, release tag, results and limitations
README	Problem, 30-second visual, architecture diagram, results table and quickstart
Colab notebook	Loads small sample, trains mini model and reproduces one rollout
Model checkpoint	Best selected model with configuration and license
Interactive demo	Parameter controls, arbitrary-time query and side-by-side animation
Technical report	6-8 pages covering methods, experiments, failures and conclusions
Demo video	60-90 seconds; problem -> controls -> comparison -> result

Recommended README opening

Project summary
ChronoPDE is a from-scratch PyTorch implementation of a continuous-time neural operator for coupled 2D reaction-diffusion dynamics. It learns a parameter-conditioned velocity field from irregular trajectory observations and integrates it at arbitrary query times. A DCT spectral backbone is evaluated against FFT-based and convolutional baselines under long-horizon and out-of-distribution shifts.

Resume bullets - fill numbers only after experiments

Use	Draft
Bullet 1	Built a boundary-aware continuous-time neural operator from scratch in PyTorch for coupled 2D reaction-diffusion PDEs, combining spline-based flow matching, DCT spectral layers and RK4 integration for arbitrary-time simulation.
Bullet 2	Benchmarked against U-Net and Fourier Neural Operator baselines across irregular observations, long-horizon rollouts and held-out physical coefficients; reduced [metric] by [X]% while profiling spectral fidelity and accuracy-compute trade-offs.

Interview defense checklist

- Derive why cosine functions satisfy homogeneous Neumann boundary conditions.
- Explain what the operator learns and how it differs from predicting the next frame.
- Explain spline derivative bias and how it affects rollout stability.
- Discuss why RK4 can improve accuracy but costs four function evaluations per step.
- State the exact OOD split and why it does not leak test information.
- Name one result that contradicted the original hypothesis and what was learned.

12. Reference Specification

Concrete defaults to prevent decision paralysis during implementation.

Experiment identifiers

Pattern	Example	Meaning
{model}-{time}-{split}-s{seed}	chronopde-irreg25-oodparam-s0	Full result identifier
Model codes	unet-ar, fno-ar, fno-ct, chronopde	Architecture and temporal formulation
Time codes	full, irreg50, irreg25	Retained observation protocol
Split codes	id, oodparam, oodic, hidden-time	Evaluation regime

Final release checklist

- 01 All tests pass in a clean environment.
- 02 README commands have been executed from a fresh clone.
- 03 Headline table includes parameter count, training time and three-seed variation.
- 04 Every chart is regenerated by a committed script.
- 05 Demo uses a frozen checkpoint and includes at least one OOD case.
- 06 Report clearly cites CFO, FNO and PDEBench and states which components were independently implemented.
- 07 No placeholder performance number remains in the resume or README.
- 08 Limitations include solver cost, spline bias, coefficient range and resolution.

Primary sources

1. Hou, Huang and Perdikaris. "CFO: Learning Continuous-Time PDE Dynamics via Flow-Matched Neural Operators." ICLR 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/8bfbf4ec87e1e331f0b1adc483b53b6b-Abstract-Conference.html
2. Li et al. "Towards Generalizable PDE Dynamics Forecasting via Physics-Guided Invariant Learning." ICLR 2026. https://proceedings.iclr.cc/paper_files/paper/2026/file/754612bde73a8b65ad8743f1f6d8ddf6-Paper-Conference.pdf
3. Majid, Sittoni and Tudisco. "Test-Time Accuracy-Cost Control in Neural Simulators via Recurrent-Depth." ICLR 2026. https://proceedings.iclr.cc/paper_files/paper/2026/file/a3b73551d62d7758bdd99fc018b167a9-Paper-Conference.pdf
4. Takamoto et al. "PDEBench: An Extensive Benchmark for Scientific Machine Learning." <https://arxiv.org/abs/2210.07182>
5. Official CFO implementation. https://github.com/shannon-hou/CFO_official
6. PDEBench repository and data-generation code. <https://github.com/pdebench/PDEBench>
7. NeuralOperator: official PyTorch implementation of neural operators. <https://github.com/neuraloperator/neuraloperator>

Final recommendation

Commit to this exact scope for the first eight weeks. The only acceptable additions are those justified by completed baseline evidence. The admissions signal will come from depth, correctness and ownership - not the number of fashionable components.